Ch4. Synchronizing Operations

Sometimes we don't just need to protect the data, we also need to synchronize actions on separate threads. One thread might need to wait for another thread to complete a task before the first thread can complete its own. In general, its common to want a thread to wait for a specific event to happen or a condition to be `true`.
It is possible to do so by periodically checking for a dedicated flag or some other similar shared data, but its far from ideal. The need to synchronize is such a common scenario that the STL provides facilities to handle it, in the form of *condition variables*, and *futures*.
These facilities are extended in the Concurrency Technical Specification, which provides additional operations for *futures*, alongside new sync facilities in the form of *latches* and *barriers*.

4.1 Waiting for an event or condition

Consider a setup where one thread is waiting for a second thread to complete a task. One way is to have it keep checking a flag in shared data (protected by a mutex) and have the second thread set the flag when it completes its task. This is wasteful on two accounts:

1. The thread consumes valuable processing time repeatedly checking the flag,
2. When the mutex is locked by the waiting thread, it can't be locked by any other thread.
   Both of these work against the thread doing the waiting: if the waiting thread is running, this limits execution resources available to run the thread being waited for, and while the waiting thread has locked the mutex protecting the flag in order to check it, the thread being waited for is unable to lock the mutex to set the flag when its done. This is akin to staying awake all night talking to the train driver: he has to drive the train more slowly because you keep distracting him, so it takes longer to get there.
   Similarly the waiting thread is consuming resources that could be used by other thread in the system, and may end up waiting longer than necessary.

A second option is have the waiting thread sleep for short periods between the checks using the `std::this_thread::sleep_for()` function.

```
bool flag;
std::mutex m;
void wait_for_flag(){
   std::unique_lock lock(m);
   while(!flag)
   {
      lock.unlock();
      std::this_thread::sleep_for(std::chrono::milliseconds(100));
      lock.lock();
   }
}
```

This is an improvement cause the thread doesn't waste processing time while it's sleeping, but it's hard to get the sleep period right. Too short a sleep, and the thread still wastes processing time checking; Too long a sleep and the thread will keep on sleeping even when the task it's waiting for is complete, introducing a delay. Only problematic in applications with time-sensitive tasks.

The third and preferred option is to use the STL facilities to wait for the event itself. The most basic mechanism for waiting for an event to be triggered by another thread is the *condition variable*.
Conceptually, a condition variable is associated with an event or other *condition*, and one or more threads can *wait* for that condition to be satisfied. When a thread has determined that the condition is satisfied, it can then *notify* one or more of the threads waiting on the condition variable in order to wake them up and allow them to continue processing.

4.1.1 Waiting with condition variables

STL provides us 2 implementations, both in the `<conditional_variable>` library header: `std::condition_variable` and `std::condition_variable_any`. In both cases, they need to work with a mutex in order to provide appropriate sync; the former is limited to working with `std::mutex` while the latter can work with anything that meets the minimal criteria for being mutex-like. Unless the flexibility can be exploited for size, performance or OS resources, `std::condition_variable` is to be used.

```
std::mutex mut;
std::queue<data_chunk> data_queue;
std::condition_variable data_cond;

void data_preparation_thread(){
   while (more_data_to_prepare()){
      data_chunk const data = prepare_data();
      {
         std::lock_guard lock(mut);
         data_queue.push(data);
      }
      data.cond.notify_one();
   }
}

void data_processing_thread(){
   while(true){
```

```
        std::unique_lock lock(mut);
        data_cond.wait(lock, () {return !data_queue.empty(); });
        data_chunk data = data_queue.front();
        data_queue.pop();
        lock.unlock();
        process(data);
        if(is_last_chunk(data))
            break;
    }
}
```

Note: Notify the condition variable after unlocking the mutex - this is so that, if the waiting thread wakes immediately, it doesn't have to block again, waiting for you to unlock the mutex.

The implementation of `wait()` first acquires the lock, and then checks the condition, (by calling the supplied lambda function) and returns if it's satisfied (lambda returns `true`). If the condition isn't satisfied (the lambda returned `false`), `wait()` unlocks the the mutex and puts the thread back in a blocked or waiting state.

This is why we need `std::unique_lock` rather than `std::lock_guard`, cause the waiting thread must unlock the mutex while it's waiting and lock it again afterward. `std::lock_guard` doesn't allow that.

Fundamentally, `std::condition_variable::wait` is *an optimization over a busy-wait*. The flexibility to unlock a `std::unique_lock` isn't just used for the `wait()` call; its also used once we have the data to process but before processing it. Processing data can potentially be a time-consuming operation, and its a bad idea to hold a lock on a mutex for longer than necessary.
Using a queue to transfer data between threads is a common scenario. Done well, the synchronization can be limited to the queue itself, which greatly reduces the possible number of synchronization problems and race conditions.

Building a thread-safe queue with condition variables (Skipping most of it)

If the waiting thread is going to wait only once, so when the condition becomes true, it will never wait on this condition variable again, a condition variable might not be the best choice of synchronization mechanisms. This is especially true if the condition being waited for is the availability of a particular piece of data. In this scenario, a *future* might be more appropriate.

4.2 Waiting for one-off events with futures

The STL models this sort of one-off event with something called a *future*. If a thread needs to wait for a specific one-off event, it somehow obtains a future representing that event. The thread can then periodically wait on the future for short periods of time to see that if the event has occurred, while performing some other task until it needs the event to have happened before it can proceed and then just wait for the future to become *ready*. A future may have data associated with it, or it may not. Once an event has happened (and the future has become *ready*), the future can't be reset.

There are two sorts of futures in the STL, implemented as two class templates in the `<future>` library header: *unique futures* (`std::future<>`) and shared futures (`std::shared_future<>`). These are modeled after `std::unique_ptr` and `std::shared_ptr`. An instance of `std::future` is the one and only instance that refers to its associated event, whereas multiple instances of `std::shared_future` may refer to the same event. In the latter case, all the instances will become *ready* at the same time, and they may all access any data associated with the event.
This associated data is the reason these are templates. The `std::future<void>` and `std::shared_future<void>` template specializations should be used where there's no associated data. Although futures are used to communicate between threads, the objects themselves don't provide synchronized accesses. If multiple thread need to access a single future object, they must protect access via a mutex or something else. But as we will soon see, multiple threads may each access their own copy of `std::_sharedfuture<>` without further synchronization, even if they all refer to the same asynchronous result.

The Concurrency TS provides extended versions of these class templates in the `std::experimental` namespace: `std::experimental::future<>` and `std::experimental::shared_future<>`. These behave identically to their counterparts in the `std` namespace, but they have additional member function to provide additional facilities.
`std::experimental` doesn't imply anything about the quality of the code, but highlights that these are non-standard classes and functions, and therefore may not have exactly the same syntax and semantics if and when they are finally adopted into a future standard.

4.2.1 Returning values from background tasks

Suppose we have a long running calculation that we expect will eventually yield a useful result but for which we don't currently need the value. We could start a new thread to perform the calculation, but that mean we have to take care of transferring the result back, because `std::thread` doesn't provide a direct mechanism of doing so. This is where the `std::async` function template (also declared in the `<future>` header)comes in.

We use `std::async` to start an *asynchronous task* for which we don't need the result right away. Rather than giving us a `std::thread` object to wait on, `std::async` returns a `std::future` object, which will eventually hold the return value of the function. When we need the value, we just call `get()` on the future, and the thread blocks until the future is ready and then returns the value.

```
#include <future>
#include <iostream>

int find_the_answer_to();
void do_other_studd();

int main(){
    std::future<int> the_answer = std::async(find_the_answer_to);
```

```
    do_other_stuff();
    std::cout << "The answer is " << the_answer.get() << std::endl;
}
```

std::async allows us to pass additional arguments to the function by adding extra arguments t the call, in the same way `std::thread` does. If the first argument is a pointer to a member function, then the second argument becomes the object on which the member function is to be applied, and the remaining arguments are to passed as parameters to the first argument.
Just as `std::thread`, if the arguments are `rvalues`, the copies are created by moving the originals. This allows the use of move-only types as both the function object and the arguments.

By default, it's up to the implementation whether `std::async` starts a new thread, or whether the task runs synchronously when the future is waited for. In most cases this is what we want, but we can specify which to use with an additional parameter to `std::async` before the function to call. This parameter is of the type `std::launch`, and can either be `std::launch::deferred` to indicate that the function call is to be deferred until either `wait()` or `get()` is called on the future, `std::launch::async` to indicate that the function must be run on its own thread, or `std::launch::deferred | std::launch::async` to let the implementation decide. The last is the default. If the function call is deferred, it may never run.

As we will see later in this chapter, using `std::async` makes it easy to divide algorithms into tasks that can be concurrently. However, its not the only way to associate a `std::future` with a task; we can also do it by wrapping the task in an instance of the `std::packaged_task<>` class template or by writing code to explicitly set the values using the `std::promise<>` class template. `std::packed_task` is a higher-level abstraction that `std::promise`, and we will start with that

4.2.2 Associating a task with a future.

`std::packaged_task<>` ties a future to a function or callable object. When the object is invoked, it calls the associated function or callable object and makes the future ready, with the return value stored as the associated data. This can be used as a building block for thread pools or other task management schemes, such as running each task on its own thread, or running them all sequentially on a particular background thread. If a large operation can be divided into self-contained sub-tasks, each of these can be wrapped in a packaged task instance, and then that instance passed to the task scheduler or thread pool. This abstracts out the details of the tasks; the scheduler just deals with packaged task instances rather than individual functions.

The template parameter for the class template is a function signature, like `void()` for a function taking no parameters with no return value, or `int(std::string&,double*)` for a function that takes a non-const reference to a std::string and a pointer to a double and returns an `int`. When we construct an instance of `std::packaged_task`, we must pass in a function or callable object that can accept the specified parameters and that returns a type that's convertible to the specified return type. The types don't have to match exactly. we can pass an int or float to a double cause they are implicitly convertible.

The return type of the specified function signature identifies the type of the `std::future<>` returned from `get_future()` member function, whereas the argument list of the function signature is used to specify the signature of the packaged task's function call operator.

4.2.3 Making (std::) promises.

When we have an application that needs to handle a lot of network connections, it's often tempting to handle each connection on a separate thread, because this can make the network communication easier to think and program. This works well for low number of connections (and thus low number of thread). Unfortunately, as the number of connections rises, this becomes less suitable; the large numbers of threads consequently consume large amounts of OS resources and potentially cause a lot of context switching, impacting performance. In extreme cases, the OS may run out of resources for running new threads before its capacity for network connections is exhausted. In applications with large numbers of network connections, its therefore common to have a small number of threads, handling the connections, with each thread dealing with multiple connections at one.

Consider one of these threads handling the connections. Data packets will come in from the various connection being handled in essentially random order, and likewise, data packets will be queued to be sent in random order. In many cases, other parts of the application will be waiting either for data to be successfully sent, or for a new batch of data to be successfully received via a specific network connection.

`std::promise<T>` provides a means of setting a value (of type `T`) that can be later be read through an associated `std::future<T>` object. A `std::promise` / `std::future` pair would provide one possible mechanism for this facility; the waiting thread could block on the future, while thread providing the data could use the promise half of the pairing to set the associated value and make the future ready.

We can obtain the `std::future` object associated with a given `std::promise` by calling the `get_future()` member function. When the value of the promise is set (using the `set_value()` member function), the future becomes ready and can be used to retrieve the stored value. If we destroy the `std::promise` without setting a value, an exception is stored instead.

If we were performing the operation in the thread that needed the result, the code could just report an error with an exception, so it would be unnecessarily restrictive to require that everything goes well just because we wanted to use `std::packaged_task` or `std::promise`. The STL therefore provides a clean way to deal with exceptions in such a scenario and allows them to be saved as part of the associated result.

4.2.4 Saving an exception for the future

Consider the following snippet of code. If we pass `-1` to the `square_root()` function, it throws an exception, and this gets seen by the caller:

```
double square_root(double x){
    if(x<0)
        throw std::out_of_range("x<0");
    return sqrt(x);
}
```

Now suppose that instead of just invoking `square_root()` from the current thread `double y = square_root(-1);` , we run the call as an asynchronous call: `std::future<double> f =std::async(square_root, -1); double y = f.get();`

It would be ideal if the behavior was exactly the same; just as `y` gets the result of the function call in either case, it would be great if the thread that called `f.get()` could see the exception too, just as it would in the single-threaded case.

Well thats exactly what happens: if the function call invoked as part of `std::async` throws an exception, that exception is stored in the future in place of a stored value, the future becomes *ready*, and a call to `get()` rethrows the stored exception.
The same happens if we wrap the function in a `std::packaged_task` -- when the task is invoked, if the wrapper function throws an exception, that exception is stored in the future in place of the result, ready to be thrown on a call to `get()` .

Naturally, `std::promise` provides the same facility, with a explicit function call. If we wish to store an exception rather than a value, we call the `set_exception()` member function rather than `set_value()` . This would typically be used in a catch block for an exception thrown as a part of the algorithm, to populate the promise with that exception

```
extern std::promise<double> some_promise;
try{
   some_promise.set_value(calculate_value());
}
catch(...){
   some_promise.set_exception(std::current_exception());
}
```

This uses `std::current_exception()` to retrieve the thrown exception; the alternative here would be to use `std::make_exception_ptr()` to store a new exception directly without throwing.

```
some_promise.set_exception( std::make_exception_ptr( std::logic_error("foo ")));
```

This is much cleaner than using a try/catch block if the type of the exception is known, and it should be used in preference; not only does it simplify the code, but it also provides the compiler with greater opportunity to optimize the code.

Another way to store an exception in a future is to destroy the `std::promise` or `std::packaged_task` associated with the future without calling either of the set functions on the promise or invoking the package task. In either case, the destructor of these two will store a `std::future_error` exception with an error code of `std::future_ercc::broken_promise` in the associated state if the future isn't already *ready*; by creating a future we make a promise to provide a value or exception, and by destroying the source of that value or exception without providing one, we break that promise. If the compiler didn't store anything in the future in this case, waiting threads could potentially wait forever.